



Universidad Espíritu Santo

Modalidad en Línea

Ingeniería en Ciencias de la Computación

Plataforma de Atención de Necesidades

Barrio Solidario

Plataforma web para coordinar asistencia comunitaria segura, trazable y accesible para adultos mayores.

Asignatura	Ingeniería de Software II
Docente	Ing. Victoria García Velásquez, Mgtr., Ph.D.(c)
Estudiantes	Alex Mendoza
	Kevin Carriel
	Mario Luzardo
	Luis Ramirez
	Luis Rodriguez
	Victor Tello

Fecha de Entrega 19 de septiembre del 2026

Guayaquil – Ecuador 2026

Identificación del documento

Campo	Información
Documento	Plan de Gestión de Cambios
Proyecto	Barrio Solidario
Versión	1.0.0
Estado	En revisión
Fecha de elaboración	15/09/2026
Propietario	Víctor Tello Bravo – Product Owner
Responsable de consolidación	Alex Mendoza Morante – Scrum Master
Revisión técnica	Kevin Carriel Bonoso – Arquitecto de software
Revisión funcional	Víctor Tello Bravo – Product Owner
Revisión de calidad	Luis Rodríguez Barrera – Calidad, pruebas y seguridad
Aprobación académica	Ing. Victoria García Velásquez
Herramientas de seguimiento	Jira, Git y Microsoft Excel
Nivel de acceso	Uso académico

Control de versiones

Versión	Fecha	Descripción del cambio	Responsable
0.1.0	15/09/2026	Creación de la estructura inicial, objetivo, alcance y principios del Plan de Gestión de Cambios.	Alex Mendoza Morante
0.2.0	15/09/2026	Definición del flujo de registro, análisis, decisión, planificación, implementación y cierre.	Alex Mendoza Morante
0.3.0	15/09/2026	Incorporación de roles, responsabilidades, clasificación, priorización e impacto funcional.	Víctor Tello Bravo
0.4.0	15/09/2026	Incorporación del análisis técnico, dependencias, arquitectura y plan de reversión.	Kevin Carriel Bonoso
0.5.0	15/09/2026	Inclusión de consideraciones de interfaz, experiencia de usuario y accesibilidad.	Mario Luzardo Fierro
0.6.0	15/09/2026	Inclusión de controles para servicios, métodos, persistencia y bases de datos.	Luis Ramírez Conejo
0.7.0	16/09/2026	Incorporación del proceso de pruebas, seguridad, evidencias y verificación posterior.	Luis Rodríguez Barrera
0.8.0	16/09/2026	Integración del formato Excel de solicitud, registro consolidado, catálogos e indicadores.	Alex Mendoza Morante
0.9.0	16/09/2026	Revisión integral de coherencia con el MVP, Jira, Git y documentación del proyecto.	Alex Mendoza Morante
1.0.0	16/09/2026	Consolidación del Plan de Gestión de Cambios para revisión y aprobación académica.	Alex Mendoza Morante

Revision y Aprobación

Nombre	Acción	Estado	Fecha
Víctor Tello Bravo	Validar el alcance, criterios funcionales, prioridades, decisiones y relación de los cambios con el MVP.	Revisado	16/09/2026
Alex Mendoza Morante	Verificar la estructura del plan, flujo de control, responsables, trazabilidad y consolidación documental.	Revisado	16/09/2026
Kevin Carriel Bonoso	Revisar el análisis técnico, arquitectura, dependencias, compatibilidad, implementación y reversión.	Revisado	16/09/2026
Mario Luzardo Fierro	Revisar los controles relacionados con interfaz, experiencia de usuario, diseño visual y accesibilidad.	Revisado	16/09/2026
Luis Ramírez Conejo	Revisar los controles relacionados con servicios, métodos, reglas de negocio, persistencia y bases de datos.	Revisado	16/09/2026
Luis Rodríguez Barrera	Validar los criterios de calidad, evaluación de riesgos, pruebas, seguridad, evidencias y cierre.	Revisado	16/09/2026
Ing. Victoria García Velásquez	Realizar la revisión académica y aprobar el Plan de Gestión de Cambios.	Pendiente de aprobación	—

Tabla de Contenidos

1.	Introducción	7
2.	Objetivo general	7
2.1.	Objetivos específicos	7
3.	Alcance del plan	7
4.	Alcance del MVP sujeto a control	8
5.	Exclusiones del plan	8
6.	Principios de gestión de cambios	9
7.	Tipos de cambios	9
7.1.	Cambio de software	9
7.2.	Cambio documental	9
7.3.	Cambio mixto	9
7.4.	Cambio de alcance	9
7.5.	Cambio correctivo	9
7.6.	Cambio adaptativo	9
7.7.	Cambio perfectivo	9
7.8.	Cambio preventivo	9
8.	Clasificación operativa	10
8.1.	Cambio normal	10
8.2.	Cambio menor	10
8.3.	Cambio urgente	10
8.4.	Cambio rechazado	10
9.	Origen de las solicitudes	10
10.	Roles y responsabilidades	10
11.	Responsabilidades del equipo	11
11.1.	Víctor Tello Bravo	11
11.2.	Alex Mendoza Morante	11
11.3.	Kevin Carriel Bonoso	11
11.4.	Mario Luzardo Fierro	11
11.5.	Luis Ramírez Conejo	12
11.6.	Luis Rodríguez Barrera	12
12.	Flujo de gestión de cambios	12
12.1.	Registro	12
12.2.	Revisión de integridad	13
12.3.	Análisis de impacto	13
12.4.	Decisión	13
12.5.	Planificación	13
12.6.	Implementación	14
12.7.	Validación	14
12.8.	Verificación posterior	14
12.9.	Cierre	14

13.	Responsables de ejecución	14
14.	Estados del proceso	15
15.	Evaluación de impacto	15
16.	Evaluación del riesgo	15
17.	Priorización	16
18.	Criterios de aprobación	16
19.	Gestión de cambios urgentes	16
20.	Plan de implementación	17
21.	Plan de reversión	17
22.	Gestión de pruebas.....	18
23.	Evidencias requeridas	18
24.	Gestión de comunicaciones	18
25.	Herramientas	19
25.1.	Microsoft Excel	19
25.2.	Jira	19
25.3.	Git	19
25.4.	Repositorio documental	19
26.	Convención de códigos.....	19
27.	Métricas e indicadores.....	20
28.	Seguimiento.....	20
29.	Control documental	20
30.	Cierre del cambio	20
31.	Auditoría y trazabilidad	21
32.	Criterios de cumplimiento del plan	21

1. Introducción

El proyecto Barrio Solidario requiere un mecanismo controlado para administrar las modificaciones que puedan presentarse durante su análisis, diseño, documentación, desarrollo y validación.

El presente Plan de Gestión de Cambios establece cómo se registrarán, analizarán, aprobarán, implementarán, probarán y cerrarán los cambios relacionados con el alcance, los requisitos, el Product Backlog, la arquitectura, el diseño, el código fuente, la base de datos, la interfaz, los controles de seguridad y la documentación del proyecto.

El plan busca evitar modificaciones informales que puedan afectar el Producto Mínimo Viable, el cronograma, la calidad o la trazabilidad de los entregables.

2. Objetivo general

Establecer el proceso, los roles, los criterios y los mecanismos de control necesarios para gestionar los cambios de software y documentación de Barrio Solidario de forma ordenada, trazable y coherente con el alcance del MVP.

2.1. Objetivos específicos

- Formalizar la presentación de solicitudes de cambio.
- Evaluar el impacto funcional, técnico, documental y académico.
- Analizar los riesgos asociados con cada modificación.
- Determinar la prioridad y urgencia de los cambios.
- Definir responsables para su evaluación e implementación.
- Evitar ampliaciones no controladas del alcance.
- Mantener la trazabilidad con requisitos, historias de usuario, Jira y Git.
- Verificar el cumplimiento de criterios de aceptación.
- Registrar evidencias de implementación y pruebas.
- Mantener actualizadas las versiones del software y los documentos.

3. Alcance del plan

El plan se aplica desde el registro de una solicitud hasta su cierre y comprende cambios relacionados con:

- Alcance del proyecto.
- Alcance del MVP.
- Requisitos funcionales y no funcionales.
- Historias de usuario.
- Criterios de aceptación.
- Product Backlog.
- Priorización de épicas, historias y tareas.
- Cronograma y planificación del Sprint.
- Arquitectura del sistema.
- Modelo de clases.
- Modelo entidad-relación.

- Diagramas BPMN.
- Código fuente.
- Servicios y métodos de backend.
- Bases de datos.
- Interfaz web.
- Accesibilidad.
- Seguridad y privacidad.
- Configuraciones.
- Wireframes.
- Matrices de calidad y riesgos.
- Documentos académicos.
- Presentaciones y evidencias.

4. Alcance del MVP sujeto a control

Durante la primera versión funcional, la gestión de cambios debe proteger principalmente los siguientes componentes:

- Portal público de Barrio Solidario.
- Navegación principal.
- Registro de usuarios.
- Inicio de sesión.
- Recuperación de contraseña.
- Cambio de contraseña.
- Control de acceso básico.
- Gestión básica del perfil.
- Panel principal correspondiente al usuario autenticado.
- Estructura inicial del proyecto Python.
- Servicios y métodos necesarios para las funciones del MVP.
- Diseño HTML y CSS de las pantallas priorizadas.
- Controles mínimos de accesibilidad, seguridad y calidad.

Cualquier solicitud que incorpore funciones como coordinación avanzada, mapas operativos completos, mensajería en tiempo real, evaluación de atenciones, moderación avanzada, analítica integral o aplicaciones móviles deberá analizarse como una posible ampliación de alcance o como una funcionalidad de una segunda versión.

5. Exclusiones del plan

No se consideran cambios sujetos al proceso completo:

- Correcciones ortográficas menores sin efecto en el contenido.
- Ajustes visuales mínimos que no alteren requisitos ni accesibilidad.
- Reorganización interna de archivos sin efecto funcional.
- Actualización de evidencias que no modifique decisiones o resultados.
- Correcciones de formato que no cambien la interpretación documental.

Estas modificaciones deben registrarse en el control de versiones cuando afecten un entregable formal.

6. Principios de gestión de cambios

La gestión de cambios se regirá por los siguientes principios:

- Todo cambio relevante debe registrarse antes de ejecutarse.
- Ningún cambio debe implementarse sin un responsable asignado.
- El cambio debe tener una justificación y un resultado esperado.
- Debe analizarse su impacto antes de aprobarlo.
- Las decisiones deben quedar documentadas.
- Los cambios deben mantenerse dentro del alcance del MVP.
- Todo cambio aprobado debe relacionarse con Jira.
- Los cambios de software deben mantener trazabilidad con Git.
- Los cambios documentales deben actualizar el control de versiones.
- Las pruebas deben responder a criterios de aceptación verificables.
- El cierre requiere evidencia de implementación y validación.
- Las modificaciones urgentes también deben documentarse.

7. Tipos de cambios

7.1. Cambio de software

Modifica código, servicios, métodos, lógica de negocio, base de datos, interfaz, configuración o infraestructura.

7.2. Cambio documental

Modifica requisitos, historias de usuario, modelos, diagramas, matrices, planes, manuales, actas o presentaciones.

7.3. Cambio mixto

Requiere modificaciones tanto en el software como en los documentos asociados.

7.4. Cambio de alcance

Agrega, elimina o modifica una funcionalidad, entregable o resultado del proyecto.

7.5. Cambio correctivo

Soluciona un defecto que impide cumplir el comportamiento esperado.

7.6. Cambio adaptativo

Ajusta el producto a una nueva condición técnica, académica o normativa.

7.7. Cambio perfectivo

Mejora una función existente, la experiencia de usuario, el rendimiento o la mantenibilidad.

7.8. Cambio preventivo

Reduce la posibilidad de errores, vulnerabilidades o problemas futuros.

8. Clasificación operativa

8.1. Cambio normal

Requiere análisis, aprobación, planificación, implementación y validación antes de cerrarse.

8.2. Cambio menor

Tiene impacto limitado, bajo riesgo y no modifica el alcance funcional. Puede seguir un proceso simplificado, pero debe conservar evidencia.

8.3. Cambio urgente

Debe ejecutarse con rapidez para solucionar un problema crítico, de seguridad o de continuidad. Puede recibir aprobación acelerada, pero debe documentarse y validarse posteriormente.

8.4. Cambio rechazado

No se implementa porque carece de justificación, excede el alcance, presenta un riesgo no aceptable o debe reservarse para una segunda versión.

9. Origen de las solicitudes

Una solicitud puede originarse por:

- Nuevo requisito.
- Defecto identificado.
- Resultado de una prueba.
- Mejora funcional.
- Riesgo detectado.
- Revisión de calidad.
- Observación de accesibilidad.
- Vulnerabilidad o problema de seguridad.
- Revisión académica.
- Cambio normativo.
- Decisión técnica.
- Dependencia de arquitectura.
- Retroalimentación del Product Owner.
- Observación de un usuario o interesado.

10. Roles y responsabilidades

Rol	Responsabilidades
Patrocinadora académica	Revisar entregables académicos y aprobar modificaciones relevantes cuando corresponda.
Product Owner	Analizar el valor funcional, proteger el alcance del MVP, priorizar y aprobar o rechazar cambios.
Scrum Master	Coordinar el flujo, verificar registros, responsables, fechas, Jira y evidencias.

Arquitecto de software	Analizar arquitectura, integraciones, dependencias, seguridad técnica y despliegue.
Frontend, UI/UX y accesibilidad	Analizar e implementar cambios de interfaz, navegación, contraste y accesibilidad.
Backend y datos	Analizar e implementar servicios, reglas de negocio, autenticación, persistencia y datos.
Calidad, pruebas y seguridad	Definir pruebas, revisar criterios, validar controles de seguridad y registrar resultados.
Solicitante	Describir la necesidad, justificarla y aportar información suficiente.
Implementador	Ejecutar el cambio aprobado y registrar evidencias.
Validador	Comprobar el cumplimiento de los criterios de aceptación.

11. Responsabilidades del equipo

11.1. Víctor Tello Bravo

Como Product Owner:

- Analiza el valor funcional.
- Determina la alineación con el MVP.
- Revisa los criterios de aceptación.
- Establece la prioridad funcional.
- Aprueba, rechaza o devuelve solicitudes.
- Valida funcionalmente los resultados.

11.2. Alex Mendoza Morante

Como Scrum Master:

- Registra o verifica las solicitudes.
- Coordina el análisis con el equipo.
- Comprueba responsables y fechas.
- Mantiene la trazabilidad con Jira.
- Da seguimiento a cambios abiertos.
- Consolida evidencias y estados.

11.3. Kevin Carriel Bonoso

Como arquitecto de software:

- Evalúa el impacto técnico.
- Identifica dependencias.
- Revisa compatibilidad y arquitectura.
- Define lineamientos técnicos.
- Revisa el plan de reversión.
- Participa en cambios de riesgo alto o crítico.

11.4. Mario Luzardo Fierro

Como responsable de frontend, UI/UX y accesibilidad:

- Evalúa cambios visuales.
- Revisa la navegación.

- Verifica contraste, legibilidad y accesibilidad.
- Implementa cambios de interfaz.
- Registra evidencias visuales.

11.5. Luis Ramírez Conejo

Como responsable de backend y datos:

- Evalúa cambios en servicios y reglas.
- Implementa métodos y lógica de negocio.
- Revisa autenticación y autorización.
- Gestiona modificaciones de datos.
- Registra evidencias técnicas.

11.6. Luis Rodríguez Barrera

Como responsable de calidad, pruebas y seguridad:

- Define escenarios de prueba.
- Valida criterios de aceptación.
- Revisa riesgos de seguridad.
- Registra defectos.
- Emite el resultado de validación.
- Recomienda el cierre o la corrección.

12. Flujo de gestión de cambios



12.1. Registro

El solicitante registra el cambio utilizando el formulario oficial. La solicitud debe contener:

- Código.
- Fecha.
- Solicitante.
- Título.
- Descripción.

- Justificación.
- Tipo y origen.
- Prioridad propuesta.
- Elementos afectados.
- Resultado esperado.

12.2. Revisión de integridad

El Scrum Master verifica que la solicitud tenga información suficiente. Si está incompleta, la devuelve al solicitante.

12.3. Análisis de impacto

El responsable funcional o técnico analiza:

- Impacto sobre el MVP.
- Alcance.
- Cronograma.
- Calidad.
- Seguridad.
- Privacidad.
- Accesibilidad.
- Compatibilidad.
- Riesgos.
- Recursos.
- Esfuerzo.
- Dependencias.
- Plan de pruebas.
- Posibilidad de reversión.

12.4. Decisión

El Product Owner, con apoyo del equipo, adopta una de las siguientes decisiones:

- Aprobar.
- Aprobar con condiciones.
- Rechazar.
- Devolver para ajuste.
- Aplazar para una segunda versión.

12.5. Planificación

Cuando el cambio es aprobado:

- Se asigna un responsable.
- Se establece una fecha objetivo.
- Se relaciona con una tarea de Jira.
- Se identifica la versión objetivo.
- Se define el plan de implementación.
- Se define el plan de pruebas.
- Se documenta el plan de reversión.

12.6. Implementación

El responsable ejecuta las actividades autorizadas, actualiza el estado y conserva evidencias.

12.7. Validación

El responsable de calidad o el validador designado comprueba los criterios de aceptación y registra el resultado.

12.8. Verificación posterior

Cuando corresponda, se comprueba que el cambio continúe funcionando después de implementarse.

12.9. Cierre

El cambio se cierra después de confirmar:

- Implementación completada.
- Criterios de aceptación cumplidos.
- Pruebas realizadas.
- Evidencias registradas.
- Documentación actualizada.
- Versión resultante identificada.
- Aprobación funcional obtenida.

13. Responsables de ejecución

Etapas	Responsable principal	Resultado
Identificación	Cualquier integrante	Necesidad identificada
Registro	Solicitante	Solicitud RFC creada
Revisión de integridad	Alex Mendoza / Scrum Master	Solicitud completa y trazable
Análisis funcional	Víctor Tello / Product Owner	Valor y relación con el MVP definidos
Análisis técnico	Kevin Carriel / Arquitecto	Impactos y solución técnica definidos
Diseño e interfaz	Mario Luzardo / Frontend y UI/UX	Cambios visuales y de accesibilidad definidos
Servicios y datos	Luis Ramírez / Backend y datos	Cambios en métodos, servicios y persistencia definidos
Calidad y riesgo	Luis Rodríguez / QA y seguridad	Riesgos, pruebas y controles establecidos
Decisión	Víctor Tello / Product Owner	Cambio aprobado, rechazado o aplazado
Planificación	Alex Mendoza y Víctor Tello	Responsable, prioridad, Sprint y fechas
Desarrollo	Mario Luzardo o Luis Ramírez	Cambio implementado
Revisión técnica	Kevin Carriel	Conformidad técnica

Pruebas	Luis Rodríguez	Resultado de validación
Validación funcional	Víctor Tello	Aceptación del resultado
Actualización documental	Responsable del documento y Scrum Master	Documento y control de versiones actualizados
Cierre	Product Owner, QA y Scrum Master	Cambio cerrado con evidencias

14. Estados del proceso

Estado	Responsable de actualización	Condición
Registrado	Solicitante o Scrum Master	Solicitud ingresada.
En análisis	Analista asignado	Evaluación en curso.
Pendiente de aprobación	Scrum Master	Análisis completo.
Aprobado	Product Owner	Cambio autorizado.
Rechazado	Product Owner	Cambio no autorizado.
Planificado	Scrum Master	Responsable y fecha definidos.
En implementación	Implementador	Ejecución iniciada.
En validación	Responsable de calidad	Cambio listo para pruebas.
Cerrado	Product Owner o responsable designado	Validación conforme.
Cancelado	Product Owner o solicitante autorizado	Cambio retirado.

15. Evaluación de impacto

Cada cambio debe evaluarse en las siguientes dimensiones:

Dimensión	Pregunta de evaluación
Alcance	¿Agrega, elimina o modifica una función o entregable?
MVP	¿Es indispensable para la primera versión funcional?
Cronograma	¿Afecta tareas, fechas o disponibilidad del equipo?
Costo	¿Requiere licencias, servicios o recursos adicionales?
Calidad	¿Modifica criterios de aceptación o pruebas?
Arquitectura	¿Afecta componentes, integraciones o estructura técnica?
Datos	¿Modifica estructuras, relaciones o información almacenada?
Seguridad	¿Afecta autenticación, permisos o protección de datos?
Privacidad	¿Modifica el tratamiento de información personal?
Accesibilidad	¿Afecta navegación, contraste, lectura o interacción?
Compatibilidad	¿Afecta navegadores, librerías o ambientes?
Documentación	¿Requiere actualizar requisitos, modelos o manuales?

16. Evaluación del riesgo

La exposición del cambio se calcula mediante:

Exposición = Probabilidad × Impacto

Exposición	Nivel	Tratamiento
1 a 4	Bajo	Puede aprobarse con controles normales.
5 a 9	Medio	Requiere medidas preventivas y seguimiento.
10 a 16	Alto	Requiere análisis técnico y aprobación expresa.
17 a 25	Crítico	Debe replantearse, reducirse o justificarse excepcionalmente.

17. Priorización

La prioridad se establecerá considerando:

- Valor para el MVP.
- Urgencia.
- Impacto funcional.
- Dependencias.
- Riesgo.
- Disponibilidad del equipo.
- Fecha del Sprint.
- Revisión académica.

Prioridad Aplicación

Highest Cambio indispensable para seguridad, funcionamiento o entrega del MVP.

High Cambio importante que debe incluirse en el Sprint.

Low Mejora que puede esperar sin afectar el resultado principal.

Lowest Cambio opcional o propuesto para una versión posterior.

18. Criterios de aprobación

Un cambio podrá aprobarse cuando:

- La solicitud esté completa.
- La necesidad esté justificada.
- El resultado esperado sea verificable.
- Se conozcan los componentes afectados.
- Los riesgos sean aceptables.
- Exista un responsable.
- Se hayan definido pruebas.
- Se encuentre dentro del MVP o exista justificación para modificarlo.
- Pueda implementarse dentro del cronograma disponible.

Un cambio podrá rechazarse o aplazarse cuando:

- Amplíe innecesariamente el alcance.
- No aporte valor al MVP.
- No tenga criterios verificables.
- Presente riesgos no controlados.
- No disponga de recursos.
- Requiera más tiempo que el disponible.
- Duplique una función existente.
- Corresponda a una segunda versión.

19. Gestión de cambios urgentes

Los cambios urgentes se utilizarán únicamente cuando exista:

- Bloqueo de una función esencial.
- Riesgo de seguridad.
- Pérdida o corrupción de información.
- Imposibilidad de iniciar sesión.
- Error que impida demostrar el MVP.
- Incumplimiento crítico de un entregable.

El procedimiento será:

1. Registrar la solicitud.
2. Clasificarla como urgente.
3. Obtener aprobación rápida del Product Owner.
4. Asignar responsable.
5. Aplicar el cambio.
6. Ejecutar pruebas mínimas.
7. Registrar evidencias.
8. Completar posteriormente el análisis y la documentación.
9. Realizar una verificación posterior.

20. Plan de implementación

Todo cambio aprobado deberá contener:

- Actividades a ejecutar.
- Responsable.
- Fecha de inicio.
- Fecha de finalización.
- Ambiente.
- Dependencias.
- Recursos.
- Evidencias esperadas.
- Criterios de aceptación.
- Plan de pruebas.
- Plan de reversión.

21. Plan de reversión

El plan de reversión debe indicar:

- Condición que activa la reversión.
- Copia o versión anterior disponible.
- Pasos para deshacer el cambio.
- Responsable.
- Tiempo estimado.
- Verificación del estado recuperado.
- Evidencia de la reversión.

Para cambios documentales, la reversión puede realizarse restaurando la versión anterior.

Para cambios de software, debe existir una rama, etiqueta, commit o respaldo que permita recuperar la versión estable.

22. Gestión de pruebas

Las pruebas deben corresponder al tipo de cambio.

Tipo de cambio	Pruebas recomendadas
Interfaz	Navegación, visualización, contraste y adaptación de pantalla.
Acceso	Credenciales válidas, inválidas, recuperación y permisos.
Backend	Reglas de negocio, respuestas, errores y validaciones.
Base de datos	Integridad, persistencia, relaciones y recuperación.
Seguridad	Autorización, sesiones, contraseñas y datos sensibles.
Documentación	Coherencia, numeración, referencias y control de versiones.
Configuración	Valores, ambientes y comportamiento esperado.

Cuando no exista un ambiente formal de QA, las pruebas podrán ejecutarse en el ambiente de desarrollo controlado, siempre que se registre la justificación y la evidencia.

23. Evidencias requeridas

Según el tipo de cambio, pueden utilizarse:

- Capturas de pantalla.
- Resultados de pruebas.
- Enlaces a Jira.
- Commits.
- Pull requests.
- Registros de ejecución.
- Archivos actualizados.
- Comparación entre versiones.
- Correos de aprobación.
- Actas de reunión.
- Firmas.
- Reportes de defectos.
- Video breve de funcionamiento.

24. Gestión de comunicaciones

Evento	Responsable	Destinatarios	Medio
Registro de solicitud	Solicitante	Scrum Master y Product Owner	Formato y Jira
Solicitud de aclaración	Scrum Master	Solicitante	Jira o mensajería del equipo
Resultado del análisis	Analista	Product Owner y Scrum Master	Jira
Decisión	Product Owner	Equipo	Jira o reunión
Inicio de implementación	Scrum Master	Equipo involucrado	Jira
Resultado de pruebas	Calidad	Product Owner y equipo	Jira y evidencia
Cierre	Scrum Master	Equipo e interesados	Jira y documento

25. Herramientas

25.1. Microsoft Excel

Se utiliza para:

- Registrar solicitudes.
- Mantener el listado consolidado.
- Aplicar listas desplegables.
- Controlar responsables y fechas.
- Visualizar estados e indicadores.

25.2. Jira

Se utiliza para:

- Planificar el cambio.
- Asignar responsables.
- Registrar tareas.
- Controlar estados.
- Adjuntar evidencias.
- Relacionar el cambio con el Sprint.

25.3. Git

Se utiliza para:

- Controlar versiones del código.
- Crear ramas.
- Registrar commits.
- Revisar modificaciones.
- Recuperar versiones anteriores.

25.4. Repositorio documental

Se utiliza para:

- Conservar archivos Word, Excel, PowerPoint y PDF.
- Mantener versiones de consulta y edición.
- Almacenar actas, modelos y evidencias.

26. Convención de códigos

Se recomienda utilizar:

RFC-AAAA-NNN

Donde:

- RFC identifica una solicitud de cambio.
- AAAA corresponde al año.
- NNN representa el número secuencial.

Ejemplos:

- RFC-2026-001
- RFC-2026-002
- RFC-2026-003

27. Métricas e indicadores

Indicador	Cálculo o interpretación
Total de solicitudes	Cantidad de cambios registrados.
Cambios aprobados	Solicitudes con decisión aprobada.
Cambios rechazados	Solicitudes no autorizadas.
Cambios cerrados	Cambios implementados y validados.
Cambios vencidos	Cambios abiertos cuya fecha objetivo ya pasó.
Tasa de aprobación	$\text{Cambios aprobados} \div \text{solicitudes evaluadas} \times 100$.
Tasa de cierre	$\text{Cambios cerrados} \div \text{cambios aprobados} \times 100$.
Cambios urgentes	Cantidad de solicitudes clasificadas como urgentes.
Cambios con rollback	Cambios que requirieron reversión.
Cambios con defectos	Cambios que presentaron resultados no conformes.
Tiempo promedio de atención	Promedio entre fecha de solicitud y cierre.

28. Seguimiento

El Scrum Master revisará periódicamente:

- Solicitudes sin responsable.
- Cambios pendientes de aprobación.
- Cambios con fechas vencidas.
- Cambios en implementación.
- Cambios sin evidencia.
- Cambios con validación no conforme.
- Cambios que requieren actualización documental.
- Cambios propuestos fuera del MVP.

El seguimiento se realizará durante las reuniones del equipo y mediante el tablero de Jira.

29. Control documental

Los cambios en documentos deben incluir:

- Actualización de la versión.
- Fecha de modificación.
- Descripción del cambio.
- Responsable.
- Revisor.
- Estado.
- Secciones afectadas.
- Archivo editable.
- Versión PDF cuando corresponda.

No debe reemplazarse una versión aprobada sin mantener registro de la modificación.

30. Cierre del cambio

Para cerrar una solicitud deben cumplirse las siguientes condiciones:

- La implementación terminó.
- Los criterios de aceptación fueron evaluados.

- La validación fue conforme.
- Las evidencias están disponibles.
- Jira está actualizado.
- Git contiene la versión correspondiente, cuando aplique.
- Los documentos afectados fueron actualizados.
- La versión resultante fue registrada.
- Se realizó la verificación posterior, cuando correspondía.
- El Product Owner aceptó el resultado funcional.

31. Auditoría y trazabilidad

Cada solicitud debe permitir responder:

- ¿Quién solicitó el cambio?
- ¿Por qué se solicitó?
- ¿Qué elemento fue afectado?
- ¿Quién lo analizó?
- ¿Qué riesgo tenía?
- ¿Quién lo aprobó?
- ¿Quién lo implementó?
- ¿Qué pruebas se realizaron?
- ¿Cuál fue el resultado?
- ¿Qué versión se generó?
- ¿Quién autorizó el cierre?
- ¿Dónde se encuentran las evidencias?

32. Criterios de cumplimiento del plan

El plan se considera correctamente aplicado cuando:

- Todos los cambios relevantes están registrados.
- Las solicitudes tienen un código único.
- Los cambios aprobados están vinculados con Jira.
- Los cambios de software tienen trazabilidad en Git.
- Los documentos mantienen control de versiones.
- Existen evidencias de implementación y pruebas.
- Los cambios fuera del MVP están aplazados o rechazados.
- Los cambios cerrados cuentan con validación.
- Los indicadores pueden calcularse con información completa.